

0.66.3. La fonction `deplace_croix()` — suivre la croix, case par case

`#include <deplace_croix>` : `deplace_croix(x, y, tuile, vitesse)` fait suivre la croix de la manette à une tuile, case par case.

1. Ce qu'il faut comprendre

Ce qu'elle fait : `deplace_croix(x, y, tuile, vitesse)` lit la **croix** et fait avancer la tuile d'une case dans ce sens, au plus une fois toutes les `vitesse` millisecondes. Elle reste dans l'écran, et n'efface que si elle a bougé : pas de clignotement.

La ligne à écrire : `#include <deplace_croix>`.

Ses arguments : la place, la tuile, la vitesse en ms écrite en clair. Seule sur sa ligne, elle range la place dans x et y. À appeler **dans la boucle**, après `image()`.

Ce qu'elle coûte : environ **330 octets**, plus la police entière.

Essaie : une vitesse de 100 : le A file.

On retrouve `deplace_croix` dans 45 leçons

[0.66.4](#), [0.67.3](#), [0.68.3](#), [0.69.3](#), [0.71](#), [0.71.1](#), [0.71.2](#), [0.72](#), [0.72.1](#), [0.72.2](#), [0.75](#), [0.75.1](#), [0.75.2](#), [0.76](#), [0.76.1](#), [0.76.2](#), [0.76.3](#), [0.76.4](#), [0.76.5](#), [0.76.6](#), [0.76.7](#), [0.76.8](#), [0.76.10](#), [0.76.11](#), [0.76.12](#), [0.76.13](#), [0.76.14](#), [0.79](#), [0.79.1](#), [0.80](#), [0.80.1](#), [0.80.2](#), [0.81](#), [0.81.1](#), [0.81.3](#), [0.83](#), [0.84](#), [0.85](#), [0.91](#), [0.91.1](#), [0.91.2](#), [0.91.3](#), [0.91.4](#), [0.91.5](#), [0.91.6](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1181 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	73 o	← la console, à l'allumage — la mise en route de la console
<code>deplace_croix()</code>	217 o	← <code>main()</code> — une fonction fournie par le langage
<code>main()</code>	32 o	← démarrage — ta fonction
VBlank	8 o	← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage	28 o	← <code>image()</code> — attend l'image suivante
AttendreVBlank	16 o	← démarrage — attend que l'écran soit libre
AttendreAcces	12 o	← <code>EffacerCases</code> — attend de pouvoir écrire une case
AdresseCase	10 o	← <code>effacer()</code> , <code>poser()</code> — trouve l'adresse d'une case de l'écran
EffacerCases	12 o	← <code>effacer()</code> — efface des cases
CopierTuiles	18 o	← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage — vide les deux cartes de l'écran
LireManette	37 o	← <code>bouton()</code> — lit les huit boutons
Tuiles	704 o	← <code>CopierTuiles</code> — la police entière

Lettres gravées : toutes (`poser()` reçoit un numéro calculé)

Pas gravé (jamais appelé) : `AvancerAirs`, `AttendreFinVBlank`, `EcrireTexte`, `EcrireNombre`, `Diviser`, `Reste`, `Multiplier`, `DecalerGauche`, `DecalerDroite`, `RangerLutins`, `InstallerTransfert`, `Hasard`, `EffacerFond`, `EffacerPanneau`,

DonneesTransfert, Notes

Fonctions du langage appelées :

bouton()	× 4	20 o sur place, dans deplace_croix()	+ LireManette (37 o)
effacer()	× 1	15 o sur place, dans deplace_croix()	+ AdresseCase (10 o), EffacerCases (12 o)
poser()	× 1	13 o sur place, dans deplace_croix()	+ AdresseCase (10 o)
image()	× 1	3 o sur place, dans main()	+ AttendreImage (28 o)
images()	× 1	2 o sur place, dans deplace_croix()	

Tes fonctions appelées :

deplace_croix()	× 1	19 o sur place, dans main()
-----------------	-----	-----------------------------

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// ---- #include <deplace_croix> : suivre la croix, case par case ----  
// La croix fait bouger le A, une case tous les quarts de seconde.  
  
#include <deplace_croix>    // deplace_croix() : la tuile suit la croix  
#include <ALPHABET>
```

2. UNE VARIABLE — X

```
uint8_t x = 9;
```

3. UNE VARIABLE — Y

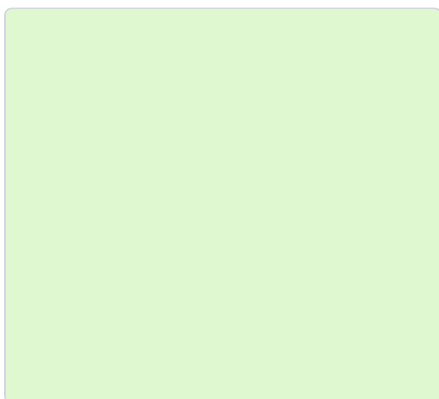
```
uint8_t y = 8;
```

4. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

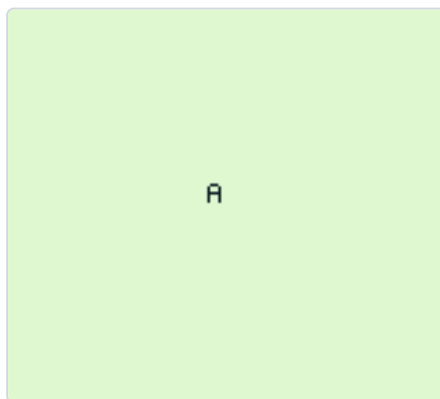
```
int main() {  
    while (true) {  
        image();  
        deplace_croix(x, y, ALPHABET[0], 250);    // x et y suivent la tuile  
    }  
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Un A au milieu ; la croix le promène, case par case.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ au départ, le A est en (9, 8)

✓ DROITE tenue : il est allé à droite ($x = 14$)

✓ et la lettre est là où dit x

Ensuite : **0.66.4. La lettre bouge avec la croix — en simple** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++